Лабораторная работа №3-4

Текст программы

Cm_timer.py

```
import time
from contextlib import contextmanager
from time import sleep
from typing import Iterator


class CmTimer1(object):
    def __init__(self) -> None:
        self.start_time: float = 0.0

    def __enter__(self) -> 'CmTimer1':
        self.start_time = time.time()
        return self

    def __exit__(self, *_) -> None:
        end_time: float = time.time()
        elapsed_time: float = end_time - self.start_time
        print(f'time: {elapsed_time}')


@contextmanager
def cm_timer_2() -> Iterator[None]:
    start_time: float = time.time()
    try:
        yield
    finally:
        end_time: float = time.time()
        elapsed_time: float = end_time - start_time
        print(f'time: {elapsed_time}')


def main():
    print('*** CmTimer1 (класс) ***')
    with CmTimer1():
        sleep(0.5)

    print('\n*** cm_timer_2 (contextlib) ***')
    with cm_timer_2():
        sleep(0.3)
```

```
if __name__ == '__main__':
    main()
```

field.py

```
from typing import List, Dict, Any, Iterator, Union


def field(items: List[Dict[str, Any]], *args: str) -> Iterator[Union[Any, Dict[str, Any]]]:
    assert len(args) > 0

    if len(args) == 1:
        key: str = args[0]
        for item in items:
            value: Any = item.get(key)
            if value is not None:
                yield value
    else:
        for item in items:
            result_dict: Dict[str, Any] = {key: item.get(key) for key in args}
            if not all(value is None for value in result_dict.values()):
                yield result_dict


def main():
    goods: List[Dict[str, Any]] = [
        {'title': 'Ковер', 'price': 2000, 'color': 'green'},
        {'title': 'Диван для отдыха', 'price': 5300, 'color': 'black'},
        {'title': 'Стул', 'color': 'white'}
    ]

    print('*** field(goods, \'title\') ***')
    for item in field(goods, 'title'):
        print(item)

    print('\n*** field(goods, \'title\', \'price\') ***')
    for item in field(goods, 'title', 'price'):
        print(item)

    goods_with_none: List[Dict[str, Any]] = [
        {'title': 'Кровать', 'price': 15000},
        {'title': None, 'price': 8000},
        {'title': 'Шкаф', 'price': None}
    ]
```

```
    print('\n*** field(goods_with_none, \'title\') ***')
    for item in field(goods_with_none, 'title'):
        print(item)


if __name__ == '__main__':
    main()
```

gen_random.py

```
import random
from typing import Iterator


def gen_random(num_count: int, begin: int, end: int) -> Iterator[int]:
    for _ in range(num_count):
        yield random.randint(begin, end)


def main():
    print('*** gen_random(5, 1, 3) ***')
    result = list(gen_random(5, 1, 3))
    print(result)


if __name__ == '__main__':
    main()
```

print_result.py

```
import functools
from typing import Callable, Any, List, Dict, TypeVar

F = TypeVar('F', bound=Callable[..., Any])


def print_result(func: F) -> F:
    @functools.wraps(func)
    def wrapper(*args: Any, **kwargs: Any) -> Any:
        result: Any = func(*args, **kwargs)

        print(func.__name__)

        if isinstance(result, list):
            for item in result:
                print(item)
        elif isinstance(result, dict):
```

```
            for key, value in result.items():
                print(f'{key} = {value}')
        else:
            print(result)

        return result
    return wrapper  # type: ignore


@print_result
def test_1() -> int:
    return 1


@print_result
def test_2() -> str:
    return 'iu5'


@print_result
def test_3() -> Dict[str, int]:
    return {'a': 1, 'b': 2}


@print_result
def test_4() -> List[int]:
    return [1, 2]


def main():
    print('!!!!!!!!')
    test_1()
    test_2()
    test_3()
    test_4()


if __name__ == '__main__':
    main()
```

process_data.py

```
import json
import sys
import os
```

```python
from typing import List, Dict, Any, Iterable

from field import field
from gen_random import gen_random
from unique import Unique
from print_result import print_result
from cm_timer import CmTimer1


def f1(arg: List[Dict[str, Any]]) -> List[str]:
    return sorted(list(Unique(field(arg, 'job-name'), ignore_case=True)), key=str.lower)


def f2(arg: List[str]) -> List[str]:
    return list(filter(lambda x: x.lower().startswith('программист'), arg))


def f3(arg: List[str]) -> List[str]:
    return list(map(lambda x: f'{x}, с опытом Python', arg))


def f4(arg: List[str]) -> List[str]:
    num_count: int = len(arg)
    salaries: Iterable[int] = gen_random(num_count, 100000, 200000)
    return list(map(lambda x: f'{x[0]}, зарплата {x[1]} руб.', zip(arg, salaries)))


@print_result
def process_data_pipeline(data: List[Dict[str, Any]]) -> List[str]:
    return f4(f3(f2(f1(data))))


def main():
    path = os.path.join(os.path.dirname(__file__), 'data_light.json')

    data: List[Dict[str, Any]] = []
    try:
        with open(path, encoding='utf-8') as f:
            data = json.load(f)
    except Exception as e:
        print(f"Ошибка загрузки данных: {e}", file=sys.stderr)
        sys.exit(1)

    with CmTimer1():
        process_data_pipeline(data)
```

```
if __name__ == '__main__':
    main()
```

unique.py

```
from typing import Iterator, Any, List, Set, Iterable


class Unique(object):
    def __init__(self, items: Iterable[Any], **kwargs: bool):
        self.ignore_case: bool = kwargs.get('ignore_case', False)
        self.items: Iterator[Any] = iter(items)
        self.seen: Set[Any] = set()

    def __next__(self) -> Any:
        while True:
            try:
                current_item: Any = next(self.items)
            except StopIteration:
                raise

            key: Any = current_item
            if self.ignore_case and isinstance(current_item, str):
                key = current_item.lower()

            if key not in self.seen:
                self.seen.add(key)
                return current_item

    def __iter__(self) -> 'Unique':
        return self


def main():
    from gen_random import gen_random

    data_int: List[int] = [1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 3]
    print('*** Unique(data_int) ***')
    print(list(Unique(data_int)))

    data_gen: Iterator[int] = gen_random(10, 1, 3)
    print('\n*** Unique(gen_random(10, 1, 3)) ***')
    print(list(Unique(data_gen)))
```

```
    data_str: List[str] = ['a', 'A', 'b', 'B', 'a', 'A', 'b', 'B']

    print('\n*** Unique(data_str) ***')
    print(list(Unique(data_str)))

    print('\n*** Unique(data_str, ignore_case=True) ***')
    print(list(Unique(data_str, ignore_case=True)))


if __name__ == '__main__':
    main()
```

sort.py

```
from typing import List


def abs_key(x: int) -> int:
    return abs(x)


data: List[int] = [4, -30, 30, 100, -100, 123, 1, 0, -1, -4]


def main():
    # Без lambda
    result: List[int] = sorted(data, key=abs_key, reverse=True)
    print('Без lambda:')
    print(result)

    # lambda
    result_with_lambda: List[int] = sorted(
        data, key=lambda x: abs(x), reverse=True)
    print('\nС lambda:')
    print(result_with_lambda)


if __name__ == '__main__':
    main()
```

Скриншоты работы приложения

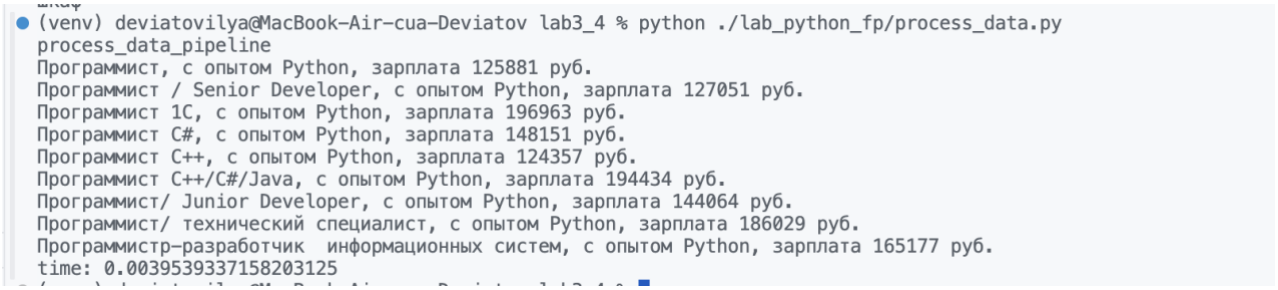

Рис. 1 – Скриншот работы приложения